File Edit View Run Kernel Settings Help Trusted

Markdown JupyterLab Python [conda env:base] * Anaconda Toolbox

Introduction to Regression

- Regression is a supervised machine learning technique used to predict a continuous numerical values based on one or more input features.
- The goal of regression is to build the relationship between independent variables and depedent variable

Example:

- Predictiong house prices
- Predicting employee salaries
- Predicting stock prices
- Predicting sales revenue

Why Regression ?

In real world businees problems, organizations often need to answer questions such as:

- How much sales incease next month?

1. Linear Regression

- Linear Regression is the simplest regression algorithm that models the relationship between input and output variables using a straight line
- It assums that a linear realtionship exists between the indepedent variable and depeddent variable.

Objective

- Find the best fitting straight line that minimizes the prediction errors.

- Find the best fitting straight line that minimizes the prediction errors.

Equation of Linear Regression

- For one feature
- y=mx+b

Where

- y = Predicted Value
- x = Input feature
- m = Slope(weight)
- b = Intercept(bias)

Example

- Salary = 5000 * Experience +20000
- If experience = 5 years
- Salary = (5000 * 5)+20000=45000

Why do we need a Cost Function ?

- Initally , the modal guesses values for slope(m) and intercept(b).
- Theses guesses are usually wrong.
- So we need a way to measure
- "How bad is our prediction ?"
- This measurement is called the Cost Function.

Mean Squared Error (MSE)

Mean Squared Error(MSE)

- The most common cost function for Linear Regression is MSE.

Formula

- MSE=Error^2+Error^2+....+Error^2/n
- Error = Actual Value-Predicted Value

Steps

1. Find the error for each prediction.
2. Square each error.
3. Add all the squared roots.
4. Divide by the total number of observations.

Example :

Actual	Predicted	Error	Error²
10	8	2	4
15	14	1	1
20	18	2	4

- MSE = (4+1+4)/3=3

Interpretation

- Low MSE = Better model
- High MSE = Low model

Gradient Descent

- Gradient Decent is a method used to find the best line by reducing the prediction error step by step.

Real time example

- Imagine you are on top of a mountain and want to reach the lowest point.

1. Take a small step downward.
2. Check if you are getting lower.
3. Keep taking the small steps down.
4. Eventually you reach the bottom.

- This is exactly how Gradient Descent work.

In Machine Learning

- Start with the random values for slope and intercept.
- Make Predictions.
- Calculate the error using cost function.
- Adjust value slightly to reduce the error.
- Repeat until the error becomes very slow.

[2]:
```
import numpy as np

#Dataset
X=np.array([1,2,3,4])
y=np.array([3,5,7,9])

#Initaial value
m=0
b=0
```

File Edit View Run Kernel Settings Help

Markdown JupyterLab Python [conda env:base] * Anaconda

```
learning_rate=0.01
epochs=1000

n=len(X)

for i in range(epochs):

    #Predictions
    y_pred=m*X+b #Current line explanation

    #Gradient
    dm=(-2/n) * np.sum(X*(y-y_pred))   #how much to change slope
    db=(-2/n) * np.sum(y-y_pred)       #how much to change intercept

    #Update anamolies
    m=m-learning_rate*dm
    b=b-learning_rate*db  #Move toward lower error

print("Slope(m):",m)
print("Intercept(b):",b)
```

```
Slope(m): 2.0048610156782556
Intercept(b): 0.9857080211211781
```

OLS(Ordinary Least Square)

- When we draw a regression line , there can be many possible lines
- OLS helps us find the best line

How?

1. Predict values using a line.
2. Compare predicted values with actual values.
3. Find the error

5. Add them together
6. Choose the line with the smallest total error.

[8]:
```
import pandas as pd
from sklearn.linear_model import LinearRegression

df=pd.DataFrame({
    "Experience":[1,2,3,4,5],
    "Salary":[30000,35000,40000,45000,50000]
})

#Input Feature
X=df[['Experience']]

#Target Variable
y=df['Salary']

# Create Model
model = LinearRegression()

# Train Model(OLS happens here)
model.fit(X,y)

#Predict Salary
predictions=model.predict(X)

#Results
print("Slope:",model.coef_[0])
print("Intercept:",model.intercept_)
print("Predicted Salaries:",predictions)
```

```
Slope: 5000.000000000001
Intercept: 24999.999999999996
Predicted Salaries: [30000. 35000. 40000. 45000. 50000.]
```

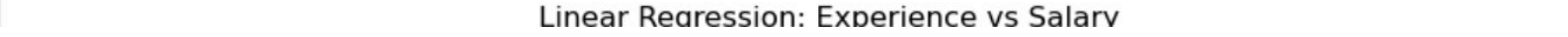
File Edit View Run Kernel Settings Help

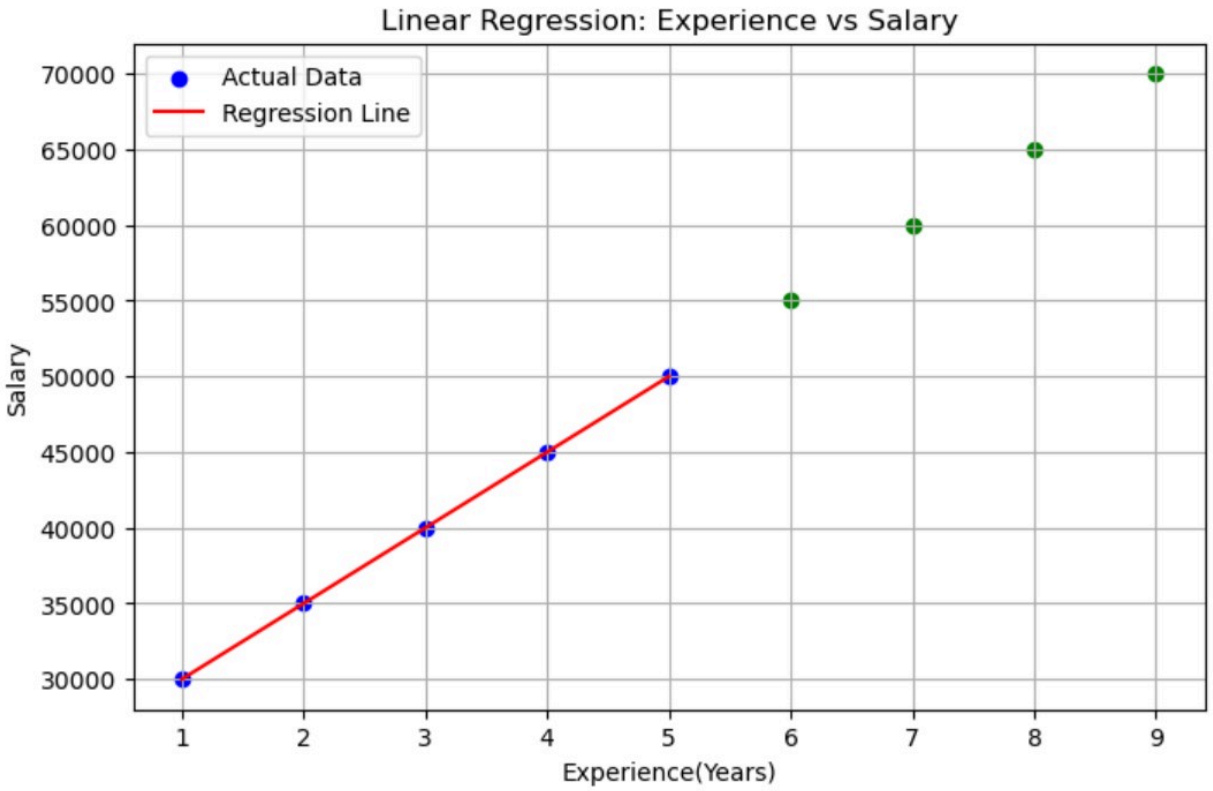
Markdown JupyterLab Python [conda env:base] * Anaconda Too

[9]:
```
new_experience=pd.DataFrame({
    'Experience':[6,7,8,9]
})

#Predict salalry for new columns
new_predictions=model.predict(new_experience)

print("Predicted Salaries for 6,7,8,9:")
print(new_predictions)
```

```
Predicted Salaries for 6,7,8,9:
[55000. 60000. 65000. 70000.]
```

[13]:
```
import matplotlib.pyplot as plt
plt.figure(figsize=(8,5))

#original data points
plt.scatter(df['Experience'],df['Salary'],color='blue',label='Actual Data')

#Regression line
plt.plot(df['Experience'],predictions,color='red',label='Regression Line')

#New predictions
plt.scatter(new_experience['Experience'],new_predictions,color='green')

#Labels
plt.xlabel("Experience(Years)")
plt.ylabel("Salary")
plt.title("Linear Regression: Experience vs Salary")
plt.legend()
plt.grid()
plt.show()
```





Multiple Regression

- Multiple regression is a regression technique that uses two or more independent variablesto predict a dependent variables.

Example

File Edit View Run Kernel Settings Help

Markdown JupyterLab Python [conda env:base] * Anaconda Toolb

Example

- Predicting a house price using

1. Area
2. Number of Bedrooms
3. House Age

- Instead of one feature , we use multiple features.

Example dataset

Area (sq ft)	Bedrooms	Age (years)	Price
1000	2	10	300000
1200	3	8	350000
1500	5	5	450000

[21]:
```
import pandas as pd
from sklearn.linear_model import LinearRegression

#Dataset
df=pd.DataFrame({
    'Area':[1000,1200,1500,1800,2000],
    'Bedrooms':[2,3,3,4,4],
    'Age':[10,8,5,4,2],
    'Price':[300000,350000,450000,500000,600000]
})
#Features
X=df[['Area','Bedrooms','Age']]
y=df['Price']
```

```
#Model
model=LinearRegression()

#Train
model.fit(X,y)

#Prediction
pred=model.predict(X)

print("Coefficient:",model.coef_)
print("Intercept:",model.intercept_)

new=pd.DataFrame({
    'Area':[2500,5488,7000,4900],
    'Bedrooms':[2,1,4,5],
    'Age':[1,45,34,23]})
new_pred=model.predict(new)
print(new_pred)
```

```
Coefficient: [   250.         -34615.38461538 -13461.53846154]
Intercept: 253846.15384615381
[ 796153.84615385  985461.53846154 1407692.30769231  996153.84615385]
```

What is Multicollinearity?

- Multicollinearity occurs when two or more indepedent variables are highly correlated with each other.

Example

Area_sqft	Area_sqm
1000	92.9
1200	111.5

File Edit View Run Kernel Settings Help

Markdown JupyterLab Python [conda env:base] * Anaconda Toolb

1200	111.5

- Both columns contain almost the same information

Prolem

- what feature are highly correlated

1. Model gets confused
2. Coefficients become unstabel
3. Interpretation becoms difficult

How to Detect Multicollinearity?

Using:

- VIF (Variance Inflation Factor)
- VIF Measures how much a feature is correlated with other features.

VIF Interpretation

VIF Value	Meaning
1	No correlation
1 - 5	Moderate correlation
> 5	High correlation
> 10	Severe correlation

File Edit View Run Kernel Settings Help

Markdown JupyterLab Python [conda env:base] * Anaconda Toolbox

```
import pandas as pd
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Features
X = df[['Area','Bedrooms','Age']]

# VIF Calculation
vif = pd.DataFrame()
vif['Feature'] = X.columns
vif['VIF'] = [
    variance_inflation_factor(X.values, i)

    for i in range(X.shape[1])
]
print(vif)
```

```
    Feature         VIF
0      Area  165.122797
1  Bedrooms  174.483505
2       Age    2.717105
```

3. Polynomial Regression

- Polynomial Regression is an extension of Linear Regression that captures non-linear relationships by adding polynomial terms.

Why Polynomial Regression?

- Not all realtionships are straight lines.

Examples:

Age vs Income Ad Spend vs Sales Temp vs Electricity Consumption

- These relationships

Degree Selection

Degree determines model complexity

- --Low Degree
    1. Simple model
    2. May underfit
- --High Degree
    1. More flexible
    2. May overfit
- --Optimal Degree
    1. Best Balance
    2. Good generalization on new data.

Advantages

- Captures non-linear patterns
- More flexible than linear Regression

Disadvantages

- Sensitive to outliers
- Higher risk of overfitting.
- Harder to interpret

[33]:
```
import pandas as pd

import numpy as np
```

File Edit View Run Kernel Settings Help

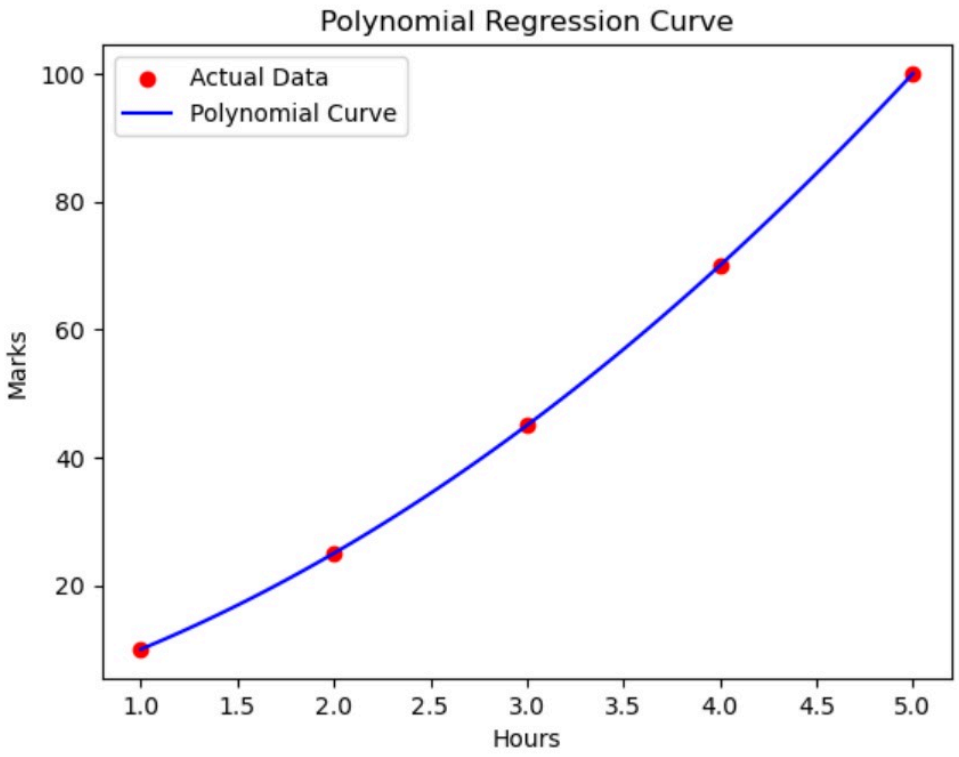
Code JupyterLab Python [conda env:base] * Anaconda

```
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Sample dataset
df = pd.DataFrame({
    'Hours': [1, 2, 3, 4, 5],
    'Marks': [10, 25, 45, 70, 100]
})

X = df[['Hours']]
y = df['Marks']

# Polynomial transformation
poly = PolynomialFeatures(degree=4)
X_poly = poly.fit_transform(X)

# Model training
model = LinearRegression()
model.fit(X_poly, y)

# Smooth curve generation (IMPORTANT STEP)
X_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
X_range = pd.DataFrame(X_range, columns=['Hours'])
X_range_poly = poly.transform(X_range)

# Predictions for curve
y_curve = model.predict(X_range_poly)

# Plotting
plt.scatter(X, y, color='red', label='Actual Data')
plt.plot(X_range, y_curve, color='blue', label='Polynomial Curve')
plt.title('Polynomial Regression Curve')
plt.xlabel('Hours')
```

File Edit View Run Kernel Settings Help

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

```
# Plotting
plt.scatter(X, y, color='red', label='Actual Data')
plt.plot(X_range, y_curve, color='blue', label='Polynomial Curve')
plt.title('Polynomial Regression Curve')
plt.xlabel('Hours')
plt.ylabel('Marks')
plt.legend()
plt.show()
```



File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

Underfit

- Occurs when the model is too simple.

Characteristics

- Poor training performance
- Poor testing performance

Goodfit

- The model captures the underlying pattern without

Overfitting

- Occurs when the model learns noise instead of actual patterns.

Characteristics

- Excellent training accuracy
- Poor testing accuracy

[39]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

X = np.array([1,2,3,4,5,6,7,8,9,10]).reshape(-1,1)
y = np.array([2,5,8,15,28,35,50,60,85,95])

# Degree 1
poly1 = PolynomialFeatures(degree=1)
```

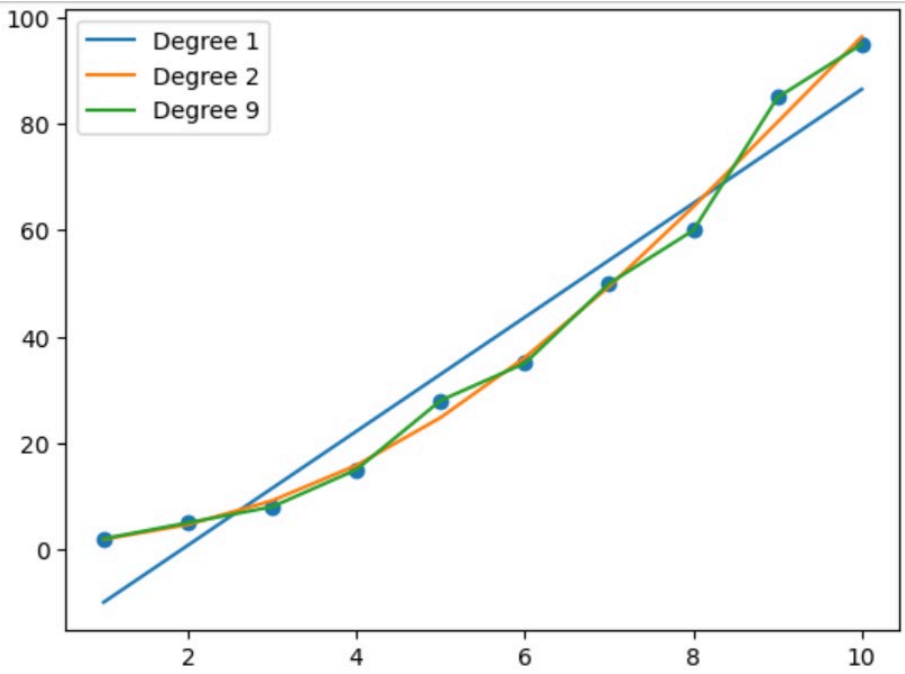
File Edit View Run Kernel Settings Help

Code JupyterLab Python [conda env:base] * Anaconda To

```
X1 = poly1.fit_transform(X)

# Degree 4
poly4 = PolynomialFeatures(degree=4)
X4 = poly4.fit_transform(X)

# Degree 9
poly9 = PolynomialFeatures(degree=9)
X9 = poly9.fit_transform(X)

model1 = LinearRegression().fit(X1,y)
model4 = LinearRegression().fit(X4,y)
model9 = LinearRegression().fit(X9,y)

plt.scatter(X,y)

plt.plot(X, model1.predict(X1), label="Degree 1")
plt.plot(X, model2.predict(X4), label="Degree 2")
plt.plot(X, model9.predict(X9), label="Degree 9")

plt.legend()
plt.show()
```

File Edit View Run Kernel Settings Help Trust

Code JupyterLab Python [conda env:base] * Anaconda Toolbox



- Degree 1 - Underfit - Straight line misses the pattern.
- Degree 4 - Goodfit - Captures the overall curve.
- Degree 9 - Overfitting - Tries to pass through every point - Learns noise instead of patter.

Regularization

- When a machine learning model becoms too complex it may memorize the training data instead of learning the underlying patterns. This problem is known as Overfitting.

- Regularization is a technique used to reduce overfitting by adding a penalty team to the model's cost function. The penalty discourages the model from assigning very large values to coefficients,helping it generalize better to unseen data.

Benefits :

- Reduce overfitting.
- Improves model generalization.
- Handles multicollinearity among features.
- Prevents very large coefficient values.
- Creates more stable and reliable model.

Types :

1. Ridge Regression(L2 Regularization)

- Ridge Regression adds the sum of squared coefficients as a penalty to term to the cost function.

Characteristics :

- Shrinks coefficient values towards 0.
- Does not make coefficient exactly 0.
- Useful when all features are important.
- Handles multicollinearity effectively.

2. Lasso Regression (L1 Regularization)

- Lasso Regression adds the sum of the absolute coefficient values as a penalty to term to the cost function.

$$\text{Cost Function} = \text{MSE} + \lambda \sum_{i=1}^{n} w_i^2$$

Characteristics :

- Shrinks coefficient towards 0.
- Can make some coefficient exactly 0.
- Performs automatic feature selection.
- Useful when many irrelevant features exist.

3. ElasticNet Regression

- Elastic combines both Ridge(L2) and Lasso(L1) penalties.

Characteristics:

- Combines advantages of Ridge and Lasso.
- Performs Feature selection.
- Handles multicollinearity.
- Suitable for datasets with many correlated features.

When to Use Each Method

Method	When to Use
Ridge (L2)	When all features are important and multicollinearity exists.
Lasso (L1)	When feature selection is required.
Elastic Net	When features are highly correlated and feature selection is also needed.

File Edit View Run Kernel Settings Help Trus

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

[41]:
```
##Ridge Code
from sklearn.linear_model import Ridge
from sklearn.datasets import make_regression

X, y = make_regression(
    n_samples=100,
    n_features=5,
    noise=20,   #unwanted data
    random_state=42   #random_state remove and once execute the value will be changing every execute
)

ridge = Ridge(alpha=1.0)   # alpha=penalty
ridge.fit(X, y)
print("Coefficients:")
print(ridge.coef_)
```

```
Coefficients:
[62.4844981  98.07649112 57.01070157 52.31272567 35.35343002]
```

[45]:
```
##Lasso Code
from sklearn.linear_model import Lasso
from sklearn.datasets import make_regression

X, y = make_regression(
    n_samples=100,
    n_features=10,
    noise=20,   #unwanted data
    random_state=42   #random_state remove and once execute the value will be changing every execute
)

lasso = Lasso(alpha=2.0)   # alpha=penalty
lasso.fit(X, y)
print("Coefficients:")
print(lasso.coef_)
```

File Edit View Run Kernel Settings Help Trus

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

```
Coefficients:
[17.88753444 54.76076704  0.         60.9632585  90.0800089  68.30461063
 81.35787563  4.03519677  2.86610797 67.99906953]
```

[47]:

```
##ElasticNet Code
from sklearn.linear_model import ElasticNet
from sklearn.datasets import make_regression

X, y = make_regression(
    n_samples=100,
    n_features=10,
    noise=20,
    random_state=42
)

elastic = ElasticNet(
    alpha=3.0,   #By Rigit it execute
    l1_ratio=0.5   #By lasso it execute 0.0 to 1.0
)

elastic.fit(X, y)

print("Coefficients:")
print(elastic.coef_)
```

```
Coefficients:
[10.65990986 22.10890939  0.49979146 25.9235125  33.08321402 34.26097045
 35.7929364  -7.32756763  4.65692898 25.41452644]
```

[ ]: